

Iniciando en SQL

Guía introductoria para estudiantes de secundario. Recorre la historia de SQL, los SGBD más usados, una mirada a MySQL y phpMyAdmin, y un paseo práctico por DDL y DML usando la base de datos de ejemplo **w3schoolsSQL**.

Índice

1. ¿Qué es SQL?
 2. Un poco de historia
 3. Los SGBD (Sistemas de Gestión de Bases de Datos)
 4. MySQL en particular
 5. phpMyAdmin: gestionar MySQL desde el navegador
 6. La base de datos de ejemplo: w3schoolsSQL
 7. DDL: Definiendo la estructura de los datos
 8. DML: Manipulando los datos
 9. Próximos pasos
-

1. ¿Qué es SQL?

SQL (Structured Query Language, o Lenguaje de Consulta Estructurado) es el lenguaje estándar para **crear, modificar, consultar y eliminar** datos en bases de datos relacionales.

En palabras simples: si una base de datos es un archivero gigante organizado en carpetas (tablas), SQL es el lenguaje con el que le pedimos al archivero que guarde, busque, modifique o saque información.

Dato importante: SQL **no** es un lenguaje de programación como PHP o JavaScript. Es un lenguaje de **consulta**: su trabajo es hablar con la base de datos.

2. Un poco de historia

- **1970:** Edgar F. Codd, un investigador de IBM, publica el modelo relacional: la idea de guardar datos en **tablas** (filas y columnas) relacionadas entre sí.
- **1974:** IBM desarrolla una primitiva versión llamada SEQUEL (Structured English Query Language).
- **1979:** Oracle (entonces RSI) lanza el primer producto comercial con SQL.
- **1986:** SQL se convierte en **estándar ANSI** (como el inglés es el idioma estándar de la aviación, SQL es el estándar de las bases de datos).
- **1987:** Se publica el estándar **ISO** de SQL.
- **1990s:** Surgen MySQL (1995), PostgreSQL (1996) y SQL Server (1989) como alternativas populares.
- **2000s hasta hoy:** SQL sigue siendo el lenguaje más usado para bases de datos en el mundo.

A pesar de tener casi 50 años, SQL no se ha quedado obsoleto: es como el inglés de las bases de datos.

3. Los SGBD (Sistemas de Gestión de Bases de Datos)

Un **SGBD** (o DBMS en inglés) es el software que permite crear, administrar y consultar bases de datos. Es el "programa" que hace que los datos se guarden y se puedan buscar.

Los más conocidos

SGBD	Tipo	¿Es gratis?	Uso típico
MySQL	Relacional	Sí	Web, aplicaciones, educación
PostgreSQL	Relacional	Sí	Proyectos grandes, Ciencia de Datos
SQLite	Relacional	Sí	Apps móviles, embebido (en el propio programa)
SQL Server	Relacional	No (tiene versión gratuita limitada)	Empresas, Microsoft
Oracle DB	Relacional	No	Bancos, grandes corporaciones

SGBD	Tipo	¿Es gratis?	Uso típico
MongoDB	No relacional	Sí	Datos flexibles (JSON), big data
MariaDB	Relacional	Sí	Fork de MySQL, alta compatibilidad

Relacional significa que los datos se organizan en **tablas** que se pueden relacionar entre sí (por ejemplo: una tabla de personas y otra de ciudades, conectadas por un campo "ciudad de nacimiento").

Para este curso usamos **MySQL** (o su versión libre, MariaDB), que es uno de los más usados en la web y en educación.

4. MySQL en particular

¿Qué es?

MySQL es un SGBD relacional de código abierto creado en 1995 por Michael "Monty" Widenius. Hoy es mantenido por Oracle.

Características principales

- **Rápido y confiable:** maneja millones de registros sin problemas.
- **Gratis:** la versión community es libre y de uso gratuito.
- **Multiplataforma:** funciona en Windows, Linux y macOS.
- **Integración natural con PHP:** casi todos los servidores web usan la combinación LAMP (Linux, Apache, MySQL, PHP).
- **Lenguaje SQL estándar:** usa el SQL que se aprende en cualquier libro o tutorial.

Datos técnicos útiles

- El archivo de datos se guarda en **C:\xampp\mysql\data** (Windows) o **/var/lib/mysql/** (Linux).
- El puerto por defecto es **3306**.
- La conexión se hace con: **host, usuario, contraseña y nombre de la base de datos**.

Conexión desde PHP (PDO)

```
$conexion = new PDO(
    "mysql:host=localhost;dbname=w3schoolsSQL;charset=utf8mb4",
    "root",    // usuario
    "",        // contraseña (vacía por defecto en XAMPP)
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);
```

PDO (PHP Data Objects) es la forma segura y moderna de conectarse a bases de datos en PHP. Siempre usá **prepared statements** (**prepare** + **execute**) para protegerte contra la inyección SQL.

5. phpMyAdmin: gestionar MySQL desde el navegador

¿Qué es?

phpMyAdmin es una aplicación web gratuita que te permite **administrar MySQL desde el navegador** sin necesidad de escribir todo a mano en la línea de comandos.

¿Para qué sirve?

- **Crear** bases de datos y tablas.
- **Ejecutar** consultas SQL (INSERT, SELECT, UPDATE, DELETE).
- **Importar** archivos **.sql** (como el de w3schools).
- **Exportar** bases de datos (para hacer respaldos).
- **Ver** la estructura de las tablas de forma visual.
- **Insertar, modificar y eliminar** datos con formularios gráficos.

¿Cómo accedés?

1. Abrí **XAMPP** y encendé **Apache** y **MySQL**.
2. En el navegador, andá a: **http://localhost/phpmyadmin**
3. A la izquierda ves las bases de datos. Hací clic en una para abrirla.
4. La pestaña **SQL** te permite pegar y ejecutar consultas directamente.

Cómo importar la base de datos de ejemplo

1. Abrí phpMyAdmin.
2. Andá a la pestaña **SQL**.
3. Pegá el contenido completo del archivo `w3schoolsSQL.sql`.
4. Hacé clic en **Continuar** (o **Go**).
5. A la izquierda verás la base `w3schoolsSQL` con sus 8 tablas.

6. La base de datos de ejemplo: w3schoolsSQL

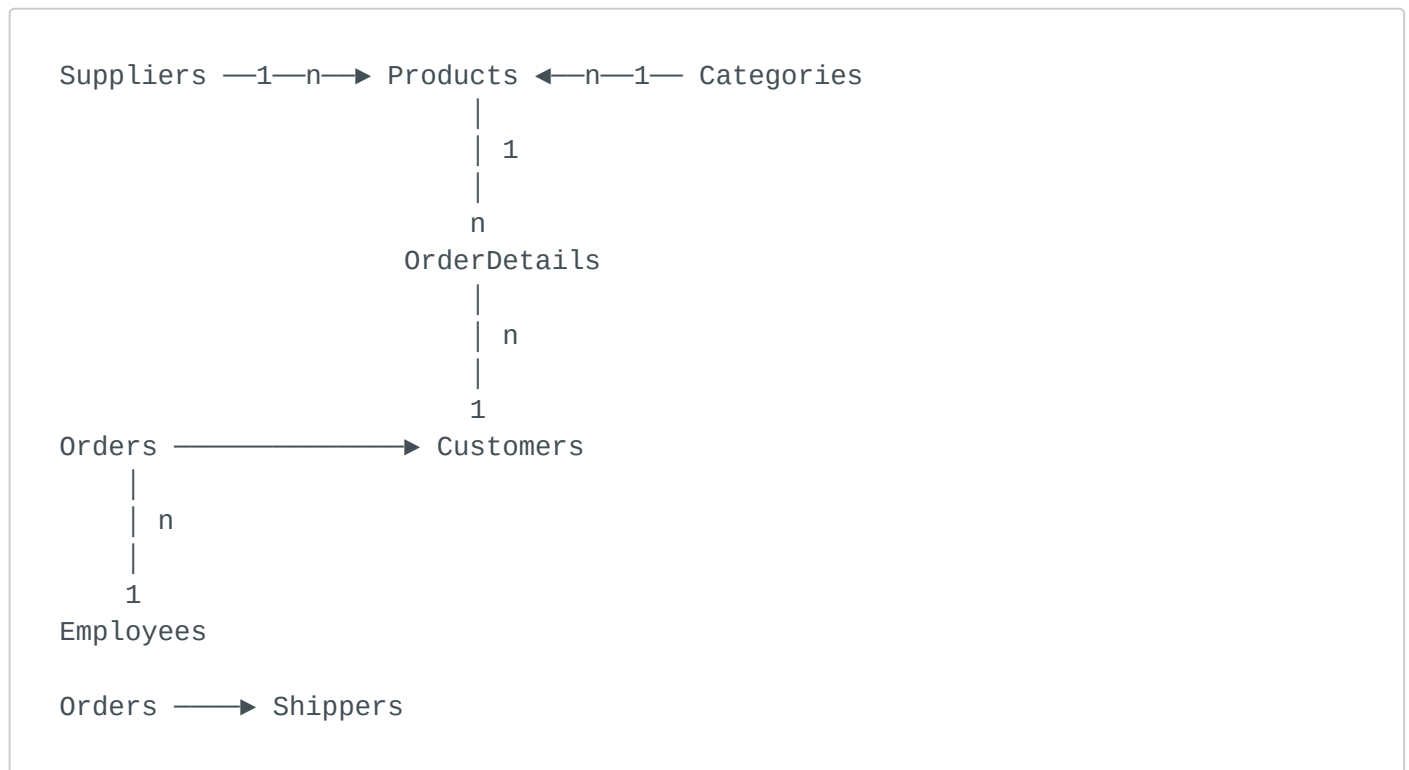
Esta base representa un **sistema de ventas** simplificado. Tiene 8 tablas que se relacionan entre sí:

Tablas y sus columnas

Tabla	Columnas principales	Qué guarda
Categories	CategoryID, CategoryName, Description	Categorías de productos (Bebidas, Lácteos, etc.)
Customers	CustomerID, CustomerName, ContactName, Address, City, PostalCode, Country	Clientes
Employees	EmployeeID, LastName, FirstName, BirthDate, Photo, Notes	Empleados
Shippers	ShipperID, ShipperName, Phone	Empresas de envío
Suppliers	SupplierID, SupplierName, ContactName, Address, City, PostalCode, Country, Phone	Proveedores
Products	ProductID, ProductName, SupplierID, CategoryID, Unit, Price	Productos
Orders	OrderID, CustomerID, EmployeeID, OrderDate, ShipperID	Pedidos realizados

Tabla	Columnas principales	Qué guarda
OrderDetails	OrderDetailID, OrderID, ProductID, Quantity	Detalle de cada pedido (qué productos y en qué cantidad)

Relaciones entre tablas



- Un **proveedor** suministra muchos **productos**.
- Un **producto** pertenece a **una categoría**.
- Un **pedido** tiene muchos **detalles** (productos pedidos).
- Un **cliente** hace muchos **pedidos**.
- Un **empleado** atiende muchos **pedidos**.
- Un **envío** lleva muchos **pedidos**.

7. DDL: Definiendo la estructura de los datos

DDL (Data Definition Language) son las sentencias SQL que **definen la estructura** de la base de datos: crear, modificar y eliminar tablas, bases de datos, índices y constraints.

Las sentencias DDL principales

CREATE DATABASE — Crear una base de datos

```
CREATE DATABASE IF NOT EXISTS w3schoolsSQL;
```

- **IF NOT EXISTS** evita un error si la base ya existe.
- Es como abrir un archivero nuevo.

CREATE TABLE — Crear una tabla

Ejemplo con la tabla **Categories** de nuestra base:

```
CREATE TABLE Categories (  
  CategoryID INT(11) NOT NULL,  
  CategoryName VARCHAR(45) DEFAULT NULL,  
  Description VARCHAR(45) DEFAULT NULL,  
  PRIMARY KEY (CategoryID)  
);
```

Desglosando cada parte:

Fragmento	Qué significa
CREATE TABLE Categories	"Creá una tabla llamada Categories"
CategoryID INT(11) NOT NULL	Columna de números enteros, obligatoria (no puede ser NULL)
CategoryName VARCHAR(45)	Texto de hasta 45 caracteres, opcional (puede ser NULL)
Description VARCHAR(45)	Texto descriptivo, opcional
PRIMARY KEY (CategoryID)	El campo que identifica de forma única cada fila

Clave primaria (PRIMARY KEY)

La clave primaria es como el **DNI** de cada fila: un número que nunca se repite y que identifica unívocamente cada registro. En esta base, cada tabla tiene su propia clave primaria.

Clave foránea (FOREIGN KEY)

La clave foránea **relaciona** dos tablas. Ejemplo en **Products**:

```
CREATE TABLE Products (  
  ProductID INT(11) NOT NULL,  
  ProductName VARCHAR(45) DEFAULT NULL,  
  SupplierID INT(11) DEFAULT NULL,  
  CategoryID INT(11) DEFAULT NULL,  
  Unit VARCHAR(255) DEFAULT NULL,  
  Price DOUBLE DEFAULT NULL,  
  PRIMARY KEY (ProductID),  
  KEY fk_Products_1_idx (CategoryID),  
  KEY fk_Products_2_idx (SupplierID),  
  CONSTRAINT fk_Products_1 FOREIGN KEY (CategoryID)  
    REFERENCES Categories(CategoryID),  
  CONSTRAINT fk_Products_2 FOREIGN KEY (SupplierID)  
    REFERENCES Suppliers(SupplierID)  
);
```

- **FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)** le dice a MySQL: *"el CategoryID de esta tabla tiene que ser un valor que exista en la tabla Categories"*.
- Esto **garantiza integridad referencial**: no se puede poner un id de categoría que no exista.

DROP TABLE — Eliminar una tabla

```
DROP TABLE IF EXISTS Categories;
```

- Borra la tabla **y todos sus datos**.
- **IF NOT EXISTS** evita error si la tabla no existe.
- Usá con cuidado: esto no tiene deshacer fácil.

ALTER TABLE — Modificar la estructura de una tabla

```
ALTER TABLE Products ADD COLUMN Stock INT DEFAULT 0;
```

- Agrega una columna nueva llamada **Stock** a la tabla **Products**.

- Otros usos de **ALTER TABLE**:
 - **MODIFY** — cambiar el tipo de un dato existente.
 - **DROP COLUMN** — quitar una columna.
 - **ADD CONSTRAINT** — agregar una clave foránea después de crear la tabla.

Tipos de datos más comunes en MySQL

Elegir el tipo de dato correcto es importante: un DNI se guarda como texto (porque puede llevar puntos), un precio como decimal, y una fecha como **DATE**. A continuación, todos los tipos organizados por categoría.

Números enteros

Tipo	Rango	Ejemplo	Uso típico
TINYINT	-128 a 127 (sin signo: 0 a 255)	1, 255	Edad, calificación, booleano (0/1)
SMALLINT	-32.768 a 32.767	1.000	Stock, cantidad de items
MEDIUMINT	-8.388.608 a 8.388.607	500.000	Contadores grandes, ID de système
INT	-2.147.483.648 a 2.147.483.647	42	IDs, cantidades, enteros generales
BIGINT	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	9999999999	IDs de tablas muy grandes, transacciones financieras

Ejemplo de CREATE TABLE:

```
CREATE TABLE Productos (  
  ProductID INT NOT NULL AUTO_INCREMENT,  
  Stock SMALLINT NOT NULL DEFAULT 0,  
  CodigoBarra BIGINT DEFAULT NULL,  
  Activo TINYINT(1) NOT NULL DEFAULT 1,  
  PRIMARY KEY (ProductID)  
);
```

Ejemplo de INSERT:

```
INSERT INTO Productos (Stock, CodigoBarra, Activo)
VALUES (150, 7891234567890, 1);
```

En un formulario HTML:

```
<label>Stock:</label>
<input type="number" name="stock" min="0" max="32767" required>

<label>Código de barras:</label>
<input type="number" name="codigo_barra" required>

<label>¿Está activo?</label>
<input type="checkbox" name="activo" value="1" checked>
<!-- El checkbox envía 1 si está marcado, nada si no. En PHP:
isset($_POST['activo']) ? 1 : 0 -->
```

Números con decimales

Tipo	Precisión	Ejemplo	Uso típico
FLOAT	~7 decimales	18.55	Datos científicos (menor precisión)
DOUBLE	~15 decimales	263.5999999	Precios, coordenadas geográficas
DECIMAL(a, b)	Exacto (a dígitos totales, b decimales)	263.50	Dinero, precios (nunca redondea)

¿Cuándo usar cuál?

- Para **dinero** usá **DECIMAL(10, 2)** siempre: **DOUBLE** puede dar resultados como **263.5999999** en vez de **263.60**.
- Para **coordenadas** (latitud, longitud) usá **DECIMAL(9, 6)** o **DOUBLE**.
- FLOAT** se usa poco hoy en día; **DOUBLE** lo reemplaza en la mayoría de los casos.

Ejemplo de CREATE TABLE:

```
CREATE TABLE Productos (
  ProductID INT NOT NULL AUTO_INCREMENT,
  ProductName VARCHAR(45) DEFAULT NULL,
  Price DECIMAL(10, 2) DEFAULT NULL, -- hasta 10 dígitos, 2 decimales
  Peso FLOAT DEFAULT NULL,
```

```
PRIMARY KEY (ProductID)
);
```

Ejemplo de INSERT:

```
INSERT INTO Productos (ProductName, Price, Peso)
VALUES ('Chais', 18.00, 0.5);
```

En un formulario HTML:

```
<label>Precio ($):</label>
<input type="number" name="price" step="0.01" min="0" placeholder="18.00"
required>
<!-- step="0.01" permite hasta 2 decimales -->

<label>Peso (kg):</label>
<input type="number" name="peso" step="0.001" min="0" placeholder="0.500">
```

`step="0.01"` le dice al navegador que el campo acepta valores con hasta 2 decimales. Si ponés `step="1"` solo acepta enteros.

Fechas y horas

Tipo	Formato	Ejemplo	Uso típico
DATE	AAAA-MM-DD	'1996-07-04'	Fecha de nacimiento, fecha de alta
TIME	HH:MM:SS	'14:30:00'	Hora de una clase, hora de apertura
DATETIME	AAAA-MM-DD HH:MM:SS	'1996-07-04 14:30:00'	Fecha y hora juntas (timestamp)
TIMESTAMP	AAAA-MM-DD HH:MM:SS	(automático)	Se llena solo con la fecha/hora actual
YEAR	AAAA	2026	Año de un modelo de auto, año de graduación

Ejemplo de CREATE TABLE:

```
CREATE TABLE Empleados (
  EmployeeID INT NOT NULL AUTO_INCREMENT,
  FirstName VARCHAR(45) NOT NULL,
  BirthDate DATE DEFAULT NULL,
  HoraEntrada TIME DEFAULT NULL,
  FechaIngreso DATETIME DEFAULT NULL,
  AnioGraduacion YEAR DEFAULT NULL,
  PRIMARY KEY (EmployeeID)
);
```

Ejemplo de INSERT:

```
INSERT INTO Empleados (FirstName, BirthDate, HoraEntrada, FechaIngreso,
AnioGraduacion)
VALUES ('Nancy', '1968-12-08', '08:30:00', '2020-03-15 09:00:00', 2015);
```

En un formulario HTML:

```
<label>Fecha de nacimiento:</label>
<input type="date" name="birth_date" required>
<!-- El navegador muestra un selector de calendario -->

<label>Hora de entrada:</label>
<input type="time" name="hora_entrada" value="08:30">

<label>Fecha y hora de ingreso:</label>
<input type="datetime-local" name="fecha_ingreso">
<!-- datetime-local muestra fecha + hora juntas -->

<label>Año de graduación:</label>
<input type="number" name="anio_graduacion" min="1900" max="2099" step="1">
<!-- No hay input type="year" en HTML, se usa number -->
```

Formato de MySQL: las fechas **siempre** se guardan como **AAAA-MM-DD** (año-mes-día), nunca como **DD/MM/AAAA**. El navegador entrega las fechas en ese formato por defecto con **type="date"**.

Textos

Tipo	Tamaño máximo	Ejemplo	Uso típico
CHAR(n)	Exactamente n caracteres	'AR' (CHAR(2))	Código de país, código postal fijo

Tipo	Tamaño máximo	Ejemplo	Uso típico
VARCHAR(n)	Hasta n caracteres	'Juan'	Nombres, emails, direcciones
TINYTEXT	Hasta 255 caracteres	'Córdoba'	Ciudad, código corto
TEXT	Hasta 65.535 caracteres	Descripción de un producto	Descripciones, notas, comentarios largos
MEDIUMTEXT	Hasta 16.777.215 caracteres	Artículo de revista	Textos muy largos, documentación
LONGTEXT	Hasta 4.294.967.295 caracteres	Libro completo	Archivos de texto enormes (se usa poco)

¿CHAR vs VARCHAR?

- **CHAR(2)** **siempre** ocupa 2 caracteres: si guardás 'AR', almacena 'AR'. Si guardás 'A', almacena 'A ' (con espacio). Es rápido para datos de longitud fija.
- **VARCHAR(45)** ocupa **solo lo necesario**: si guardás 'Juan', ocupa 4 caracteres + 1 byte de longitud. Es más eficiente para textos variables.

Ejemplo de CREATE TABLE:

```
CREATE TABLE Productos (
  ProductID INT NOT NULL AUTO_INCREMENT,
  Codigo CHAR(3) DEFAULT NULL, -- ej: 'CH1'
  ProductName VARCHAR(45) NOT NULL, -- nombre corto
  Unit VARCHAR(255) DEFAULT NULL, -- empaque
  Description TEXT DEFAULT NULL, -- descripción larga
  PRIMARY KEY (ProductID)
);
```

Ejemplo de INSERT:

```
INSERT INTO Productos (Codigo, ProductName, Unit, Description)
VALUES ('CH1', 'Chais', '10 boxes x 20 bags',
'Delicate light flavored tea blended with a hint of rose.');
```

En un formulario HTML:

```
<!-- CHAR / VARCHAR corto -->
<label>Código (3 caracteres):</label>
<input type="text" name="codigo" maxlength="3" pattern="[A-Z0-9]{3}" required
placeholder="CH1">
```

```

<!-- maxlength limita la cantidad de caracteres en el formulario -->
<!-- pattern valida que sean 3 caracteres alfanuméricos en mayúscula -->

<!-- VARCHAR mediano -->
<label>Nombre del producto:</label>
<input type="text" name="product_name" maxlength="45" required
        placeholder="Chais">

<!-- VARCHAR largo -->
<label>Unidad de venta:</label>
<input type="text" name="unit" maxlength="255"
        placeholder="10 boxes x 20 bags">

<!-- TEXT (textarea para textos largos) -->
<label>Descripción:</label>
<textarea name="description" rows="6" cols="50"
        placeholder="Descripción del producto..."></textarea>

```

Nota: el **textarea** de HTML es el equivalente al **TEXT** de MySQL. Para textos muy largos (**MEDIUMTEXT**, **LONGTEXT**) también se usa **textarea**, pero en la práctica rara vez se necesitan esos tamaños en formularios web.

Valores lógicos (booleanos)

MySQL no tiene un tipo **BOOLEAN** como tal. Internamente, **BOOLEAN** es un alias de **TINYINT(1)**:

- **1** = verdadero (true)
- **0** = falso (false)

Tipo	Valores	Ejemplo	Uso típico
BOOLEAN (alias)	0 o 1	TRUE / FALSE	¿Está activo? ¿Es visible? ¿Pagó?
TINYINT(1)	0 a 127 (pero se usa 0/1)	1	Mismo que BOOLEAN

Ejemplo de CREATE TABLE:

```

CREATE TABLE Usuarios (
    UserID      INT          NOT NULL AUTO_INCREMENT,
    Username    VARCHAR(50)  NOT NULL,
    Activo      BOOLEAN      NOT NULL DEFAULT TRUE,
    Admin       BOOLEAN      NOT NULL DEFAULT FALSE,
    PRIMARY KEY (UserID)
);

```

Ejemplo de INSERT:

```
INSERT INTO Usuarios (Username, Activo, Admin)
VALUES ('juan123', TRUE, FALSE);
-- Equivalente a:
INSERT INTO Usuarios (Username, Activo, Admin)
VALUES ('juan123', 1, 0);
```

En un formulario HTML:

```
<label>
  <input type="checkbox" name="activo" value="1" checked>
  Usuario activo
</label>

<label>
  <input type="checkbox" name="admin" value="1">
  Es administrador
</label>
```

En PHP (al recibir los datos):

```
$activo = isset($_POST['activo']) ? 1 : 0;
$admin  = isset($_POST['admin'])  ? 1 : 0;
```

- Si el checkbox está marcado, `$_POST['activo']` tiene valor `'1'`.
- Si **no** está marcado, **no se envía nada** (no aparece en `$_POST`). Por eso se usa `isset()`.

Resumen rápido: ¿qué tipo elijo?

Dato	Tipo recomendado	¿Por qué?
ID	INT o BIGINT	Números que se autoincrementan
Nombre, email	VARCHAR(100-255)	Texto con límite razonable
Código postal	VARCHAR(10)	Puede tener letras (ej: 'T2F 8M4')
DNI / CUIT	VARCHAR(20)	Puede llevar puntos o guiones
Descripción	TEXT	Texto largo sin límite fijo
Precio	DECIMAL(10, 2)	Exacto, sin errores de redondeo

Dato	Tipo recomendado	¿Por qué?
Fecha de nacimiento	DATE	Solo fecha, sin hora
Fecha de registro	DATETIME	Fecha + hora del momento
¿Está activo?	BOOLEAN	Solo true/false
Stock	SMALLINT	Entero corto, no necesita_billones

Engine y charset

En los scripts de esta base verás al final de cada `CREATE TABLE`:

```
ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

- **InnoDB**: el motor de almacenamiento. Soporta claves foráneas y transacciones (operaciones que se pueden deshacer).
- **latin1**: el conjunto de caracteres. Para textos con tildes y ñ se recomienda `utf8mb4`.

8. DML: Manipulando los datos

DML (Data Manipulation Language) son las sentencias que **trabajan con los datos** dentro de las tablas: insertar, consultar, modificar y eliminar registros.

INSERT — Agregar datos

Sintaxis básica:

```
INSERT INTO tabla (columna1, columna2, columna3)
VALUES (valor1, valor2, valor3);
```

Ejemplo real con la tabla `Shippers`:

```
INSERT INTO Shippers (ShipperID, ShipperName, Phone)
VALUES (1, 'Speedy Express', '(503) 555-9831');
```

Se pueden insertar varias filas de una sola vez:

```
INSERT INTO Shippers (ShipperID, ShipperName, Phone)
VALUES
  (1, 'Speedy Express', '(503) 555-9831'),
  (2, 'United Package', '(503) 555-3199'),
  (3, 'Federal Shipping', '(503) 555-9931');
```

Consejo: si el campo es **AUTO_INCREMENT** (como los IDs en PHP), no hace falta especificarlo: MySQL le asigna el siguiente número automáticamente.

SELECT — Consultar datos

La sentencia más usada en SQL. Es como preguntarle a la base de datos.

Traer todos los datos de una tabla:

```
SELECT * FROM Categories;
```

Traer solo algunas columnas:

```
SELECT CategoryName, Description FROM Categories;
```

Filtrar con WHERE:

```
SELECT * FROM Products WHERE Price > 50;
```

Ordenar con ORDER BY:

```
SELECT * FROM Products ORDER BY Price DESC;
```

- **ASC** = ascendente (menor a mayor). Es el valor por defecto.
- **DESC** = descendente (mayor a menor).

Limitar resultados con LIMIT:

```
SELECT * FROM Products ORDER BY Price DESC LIMIT 5;
```

- Trae solo los 5 productos más caros.

Buscar texto con LIKE:

```
SELECT * FROM Customers WHERE CustomerName LIKE '%ch%';
```

- **%** es un **comodín**: representa "cualquier cosa". **%ch%** busca nombres que contengan "ch" en cualquier posición.

Contar filas con COUNT:

```
SELECT COUNT(*) FROM Orders;
```

- Devuelve cuántos pedidos hay en total.

Agrupar con GROUP BY:

```
SELECT CategoryID, COUNT(*)  
FROM Products  
GROUP BY CategoryID;
```

- Cuenta cuántos productos hay por cada categoría.

UPDATE — Modificar datos existentes

```
UPDATE Products  
SET Price = 20  
WHERE ProductID = 1;
```

- Cambia el precio del producto con ProductID = 1 a 20.
- **Sin el WHERE**, se cambiaría el precio de **todos** los productos. Siempre filtrá con **WHERE**.

DELETE — Eliminar datos

```
DELETE FROM OrderDetails WHERE OrderDetailID = 1;
```

- Elimina el detalle de pedido con ese id.
- Sin el **WHERE**, se eliminan **todas** las filas de la tabla. Usá extrema precaución.

Resumen visual de DML

Acción	Sentencia	Ejemplo
Crear (datos)	INSERT INTO	INSERT INTO Categories VALUES (9, 'Snacks', 'Frituras');
Leer (datos)	SELECT	SELECT * FROM Products WHERE Price < 10;
Actualizar (datos)	UPDATE ... SET	UPDATE Products SET Price = 25 WHERE ProductID = 3;
Eliminar (datos)	DELETE FROM	DELETE FROM Customers WHERE CustomerID = 99;

Tipos de consultas: de una tabla a varias

Las consultas SQL se pueden clasificar por **cuántas tablas involucran** y **cómo se combinan** los datos. Usaremos la base w3schoolsSQL para ver cada tipo con ejemplos reales.

1. Consultas sobre una sola tabla

Son las más simples: todo el dato viene de **una tabla**. Son la base de todo lo demás.

Ejemplo: todos los productos con precio mayor a 50

```
SELECT ProductName, Price  
FROM Products  
WHERE Price > 50  
ORDER BY Price DESC;
```

Resultado parcial:

ProductName	Price
Côte de Blaye	263.50
Thüringer Rostbratwurst	123.79
Sir Rodney's Marmalade	81.00

Ejemplo: clientes de un país específico

```
SELECT CustomerName, ContactName, City
FROM Customers
WHERE Country = 'Mexico';
```

Ejemplo: empleados nacidos después de 1960

```
SELECT FirstName, LastName, BirthDate
FROM Employees
WHERE BirthDate > '1960-01-01'
ORDER BY BirthDate;
```

Regla: cuando todo lo que necesitas está en una tabla, no busques complicar: una sola consulta con **WHERE** alcanza.

2. Consultas sobre varias tablas SIN JOIN

A veces necesitas datos de dos tablas, pero **no están relacionadas con una clave foránea** directa, o simplemente quieres combinar los resultados de forma independiente.

Ejemplo: qué clientes hicieron pedidos (sin ver el detalle)

```
SELECT Customers.CustomerName, Orders.OrderID, Orders.OrderDate
FROM Customers, Orders
WHERE Customers.CustomerID = Orders.CustomerID;
```

Esto es un **INNER JOIN implícito**: la coma entre las tablas en el **FROM** y el **WHERE** que iguala los IDs es la forma antigua de relacionar tablas. Funciona igual que un **JOIN**, pero es menos claro.

Ejemplo: todos los clientes y todos los pedidos (incluso los que no tienen)

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers, Orders;
```

Esto genera un **PRODUCTO CARTESIANO** (o cross join implícito): cada fila de **Customers** se combina con cada fila de **Orders**. Si hay 100 clientes y 830 pedidos, el resultado tendría $100 \times 830 = \mathbf{83.000}$ filas. Casi nunca es lo que querés.

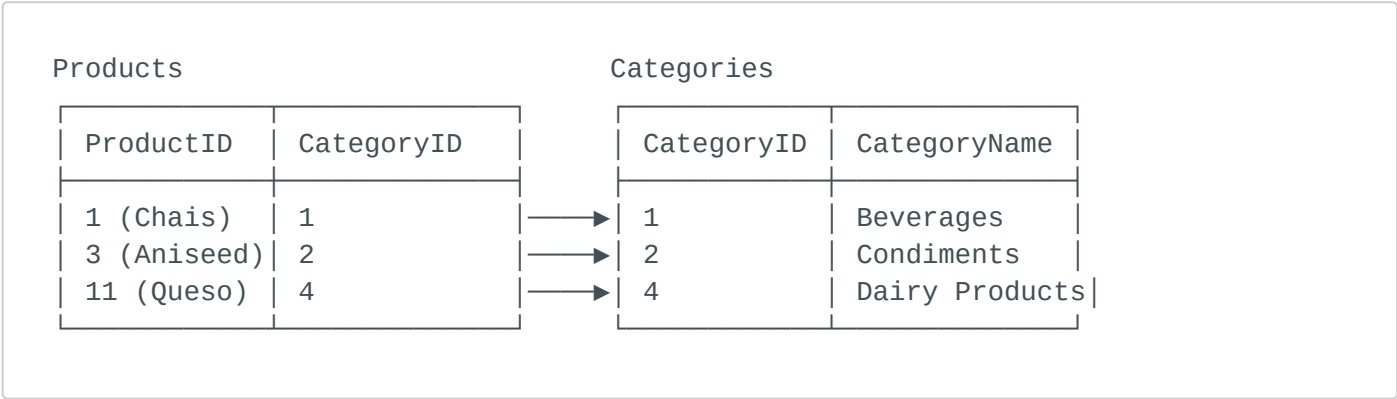
Consejo: evitá las consultas con coma. Usá **JOIN** explícito: es más legible y evita productos cartesianos accidentales.

3. Consultas con JOIN (combinar tablas relacionadas)

JOIN es la forma correcta y moderna de combinar datos de dos o más tablas relacionadas.

¿Cómo funciona JOIN?

Imaginá dos pilas de tarjetas: una de productos y otra de categorías. **JOIN** es como tomar cada producto y buscar la categoría que le corresponde por su **CategoryID**.



Tipos de JOIN:

Tipo	Qué hace	Cuándo usarlo
INNER JOIN	Solo muestra filas que tienen coincidencia en ambas tablas	Lo más común: solo productos con categoría asignada
LEFT JOIN	Muestra todas las filas de la tabla izquierda, aunque no tengan coincidencia	Querés ver todos los productos, aunque algunos no tengan categoría
RIGHT JOIN	Muestra todas las filas de la tabla derecha	Querés ver todas las categorías, aunque no tengan productos

Tipo	Qué hace	Cuándo usarlo
CROSS JOIN	Producto cartesiano (cada fila con cada fila)	Casos muy específicos (calendarios, combinaciones)

Ejemplo INNER JOIN: productos con el nombre de su categoría

```
SELECT p.ProductName, p.Price, c.CategoryName
FROM Products p
INNER JOIN Categories c ON p.CategoryID = c.CategoryID;
```

Resultado parcial:

ProductName	Price	CategoryName
Chais	18.00	Beverages
Chang	19.00	Beverages
Aniseed Syrup	10.00	Condiments
Queso Cabrales	21.00	Dairy Products

Ejemplo LEFT JOIN: todos los productos, incluso los sin categoría

```
SELECT p.ProductName, p.Price, c.CategoryName
FROM Products p
LEFT JOIN Categories c ON p.CategoryID = c.CategoryID;
```

Si algún producto tuviera **CategoryID = NULL** o un id que no existe en **Categories**, aparecería con **CategoryName = NULL**.

Ejemplo con tres tablas: pedido con cliente y empleado

```
SELECT o.OrderID, c.CustomerName, e.FirstName, e.LastName, o.OrderDate
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID
INNER JOIN Employees e ON o.EmployeeID = e.EmployeeID
WHERE o.OrderDate > '1996-07-10'
ORDER BY o.OrderDate;
```

Resultado parcial:

OrderID	CustomerName	FirstName	LastName	OrderDate
10250	Bólido Comidas preparadas	Margaret	Peacock	1996-07-08
10251	Chop-suey Chinese	Janet	Leverling	1996-07-08

Alias: **p**, **c**, **e** son apodos para las tablas. Sin tendrías que escribir **Products.ProductName** en vez de solo **p.ProductName**.

Ejemplo: producto con su proveedor y categoría (tres tablas)

```
SELECT p.ProductName, p.Price, s.SupplierName, c.CategoryName
FROM Products p
INNER JOIN Suppliers s ON p.SupplierID = s.SupplierID
INNER JOIN Categories c ON p.CategoryID = c.CategoryID
WHERE p.Price > 50
ORDER BY p.Price DESC;
```

4. Subconsultas (consultas anidadas)

Una **subconsulta** es una consulta SQL adentro de otra. Es como preguntar algo y usar la respuesta para preguntar otra cosa.

Ejemplo: productos más caros que el promedio

```
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products);
```

¿Qué pasa acá?

1. La **subconsulta** (**SELECT AVG(Price) FROM Products**) se ejecuta primero: calcula el precio promedio de todos los productos.
2. La **consulta principal** usa ese resultado para filtrar: "traeme los productos cuyo precio sea mayor a ese promedio".

Ejemplo: clientes que al menos hicieron un pedido

```
SELECT CustomerName
FROM Customers
WHERE CustomerID IN (SELECT DISTINCT CustomerID FROM Orders);
```

1. La subconsulta trae los **CustomerID** únicos de **Orders**.
2. La consulta principal busca los clientes cuyo **CustomerID** esté en esa lista.

Ejemplo: el producto más caro de cada categoría (con subconsulta correlacionada)

```
SELECT p.ProductName, p.Price, c.CategoryName
FROM Products p
INNER JOIN Categories c ON p.CategoryID = c.CategoryID
WHERE p.Price = (
    SELECT MAX(p2.Price)
    FROM Products p2
    WHERE p2.CategoryID = p.CategoryID
);
```

Una **subconsulta correlacionada** se ejecuta **una vez por cada fila** de la consulta externa. Es más lenta pero muy poderosa.

Ejemplo: empleados que atendieron pedidos de clientes de Germany

```
SELECT DISTINCT e.FirstName, e.LastName
FROM Employees e
WHERE e.EmployeeID IN (
    SELECT o.EmployeeID
    FROM Orders o
    WHERE o.CustomerID IN (
        SELECT c.CustomerID
        FROM Customers c
        WHERE c.Country = 'Germany'
    )
);
```

Acá hay **dos subconsultas anidadas**: primero se buscan los clientes de Germany, luego los pedidos de esos clientes, y finalmente los empleados que atendieron esos pedidos.

Regla: las subconsultas van entre **paréntesis** y se ejecutan **antes** que la consulta externa.

5. Consultas sumarias (agregaciones)

Las consultas sumarias **resumen datos**: en vez de traer filas individuales, traen **un número** que representa un cálculo sobre varios registros.

Funciones de agregación principales:

Función	Qué hace	Ejemplo
<code>COUNT(*)</code>	Cuenta cuántas filas hay	¿Cuántos productos hay?
<code>COUNT(columna)</code>	Cuenta filas where la columna no es NULL	¿Cuántos productos tienen categoría?
<code>SUM(columna)</code>	Suma todos los valores	¿Cuál es el total de ventas?
<code>AVG(columna)</code>	Calcula el promedio	¿Cuál es el precio promedio?
<code>MIN(columna)</code>	Trae el valor mínimo	¿Cuál es el producto más barato?
<code>MAX(columna)</code>	Trae el valor máximo	¿Cuál es el producto más caro?

Ejemplo: resumen general de productos

```
SELECT
  COUNT(*)           AS TotalProductos,
  MIN(Price)         AS PrecioMinimo,
  MAX(Price)         AS PrecioMaximo,
  ROUND(AVG(Price), 2) AS PrecioPromedio,
  SUM(Price)         AS SumaTotalPrecios
FROM Products;
```

Resultado:

TotalProductos	PrecioMinimo	PrecioMaximo	PrecioPromedio	SumaTotalPrecios
77	2.50	263.50	28.87	2222.74

`ROUND(AVG(Price), 2)` redondea el promedio a 2 decimales para que se vea prolijo.

Ejemplo: cuántos productos hay por categoría

```
SELECT c.CategoryName, COUNT(*) AS CantidadProductos
FROM Products p
INNER JOIN Categories c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryName
ORDER BY CantidadProductos DESC;
```

Resultado:

CategoryName	CantidadProductos
Confections	13
Beverages	12
Condiments	12
Dairy Products	10
Seafood	12
Grains/Cereals	7
Meat/Poultry	6
Produce	5

GROUP BY agrupa las filas que tienen el mismo valor en la columna indicada y luego la función de agregación (**COUNT**) se aplica **sobre cada grupo**.

Ejemplo: total de unidades vendidas por producto (solo los que superan 100 unidades)

```
SELECT p.ProductName, SUM(od.Quantity) AS TotalUnidades
FROM OrderDetails od
INNER JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductName
HAVING TotalUnidades > 100
ORDER BY TotalUnidades DESC;
```

Resultado parcial:

ProductName	TotalUnidades
Guaraná Fantástica	153
Alice Mutton	145
Rössle Sauerkraut	140

¿WHERE vs HAVING?

	WHERE	HAVING
Se usa para	Filtrar filas antes de agrupar	Filtrar grupos después de agrupar
Va antes de	GROUP BY	GROUP BY

	WHERE	HAVING
Funciones de agregación	No puede usar <code>COUNT(*)</code> , <code>SUM()</code> , etc.	Puede usarlas
Ejemplo	<code>WHERE Price > 10</code>	<code>HAVING COUNT(*) > 5</code>

Ejemplo completo: países con más de 5 clientes

```
SELECT Country, COUNT(*) AS CantidadClientes
FROM Customers
GROUP BY Country
HAVING CantidadClientes > 5
ORDER BY CantidadClientes DESC;
```

Ejemplo: clientes que gastaron más de 5000 en total (usando JOIN + GROUP BY + HAVING)

```
SELECT c.CustomerName, SUM(p.Price * od.Quantity) AS TotalGastado
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID
INNER JOIN OrderDetails od ON o.OrderID = od.OrderID
INNER JOIN Products p ON od.ProductID = p.ProductID
GROUP BY c.CustomerName
HAVING TotalGastado > 5000
ORDER BY TotalGastado DESC;
```

Resumen: ¿cuándo uso cada tipo de consulta?

Tipo	Ejemplo en la vida real	Sentencia clave
Una tabla	"¿Cuántos productos cuestan más de \$50?"	<code>SELECT ... WHERE</code>
Varias tablas (sin JOIN)	"Clientes y sus pedidos" (forma antigua)	<code>FROM t1, t2 WHERE t1.id = t2.id</code>
Con JOIN	"Productos con el nombre de su categoría"	<code>JOIN ... ON</code>
Subconsulta	"Productos más caros que el promedio"	<code>WHERE x > (SELECT ...)</code>
Sumaria	"Cuántos productos hay por categoría"	<code>GROUP BY + COUNT/SUM/AVG</code>

Consejo: empezá simple. Si la consulta se puede resolver con una tabla, no uses dos. Si necesitás dos tablas, usá **JOIN**. Si necesitás filtrar por un cálculo, usá subconsulta o **HAVING**.

9. Próximos pasos

Con lo que viste en esta guía ya podés:

- Entender qué hace SQL y por qué es importante.
- Crear bases de datos y tablas con DDL.
- Insertar, consultar, modificar y eliminar datos con DML.
- Usar phpMyAdmin para explorar y practicar.

Para seguir aprendiendo

- Practicá con la base **w3schoolsSQL** en phpMyAdmin: probá las consultas de esta guía y modificá los filtros.
- Consultá la documentación oficial: <https://dev.mysql.com/doc/>
- Repasá los PDFs de la carpeta **sql/** para profundizar en DDL y DML.
- Avanzá con las etapas del proyecto **etapa2-BD** y **etapa3-BD-tablas**, donde verás cómo PHP se conecta a MySQL para crear aplicaciones web completas.

La mejor forma de aprender SQL es practicando. Abrí phpMyAdmin, escribí consultas, rompé cosas (en una base de prueba), y volvé a intentar.
